

Call Center Quality Score and Monitoring Automation Using Machine Learning

Prepared by : **Hüseyin Bilgili**
 Student No : **090203760**
 Submission Date : **January 14, 2026**

Course : **MAT 4902E**
 Supervisor : **ASSIS. PROF. DR. İZZET GÖKSEL**

Abstract

Today, organizations with large customer bases allocate significant resources to enhance their customer service departments. This project describes an automatic system for scoring the quality of Turkish call center conversations from 0 to 100. The system transcribes the call audio with WhisperX , separates speakers with pyannote.audio , maps them to representative and customer roles and scores the conversation with a fine tuned BERTurk regression model . It also adds rule based scores to decrease model error and improves the learned score. I compared seven different modeling approaches on the same test set. The best result has come from the fine-tuned BERTurk model with a mean absolute error of 7.35 and an R2 of 0.876. The fine tuned model design is compared by the combination of Maximum Entropy and Log Likelihood Ratio methods. The project is showed by interactive web application using streamlit.

Table of Contents

Abstract	2
Table of Contents	2

1. Introduction.....	5
2. Background and Related Work.....	6
2.1 Speech-to-Text and Diarization	6
2.2 Text Representation	6
2.3 Automated Quality Monitoring	7
2.4 Toward an Explainable, Hybrid System	7
3. Dataset and Preparation	8
3.1 Dataset Description	8
3.2 Data Augmentation	9
3.3 Preprocessing and Speaker Mapping	9
3.4 Quality Score and Label Definition	9
4. System Architecture	10
4.1 Speech-to-Text and Diarization	10
4.2 Automatic Speaker-Role Assignment	10
4.3 NLP Analysis with BERTurk	10
4.4 Rule-Based Engine	11
4.5 Hybrid Scoring Engine.....	12
4.6 Implementation and Engineering Challenges.....	12
5. Modeling Approaches	13
5.1 Baseline Regressors	13
5.2 BERTurk Fine-Tuned Regression	13
5.3 Maximum-Entropy Classifier	13
5.4 Log-Likelihood-Ratio Monitor.....	14
5.5 MaxEnt + LLR Hybrid.....	14
5.6 Implementation Details	14
6. Experiments and Results	15
6.1 Experimental Setup	15
6.2 Evaluation Metrics.....	15
6.2.1 Mean Absolute Error (MAE)	15

6.2.2 Root Mean Squared Error (RMSE)	16
6.2.3 Coefficient of Determination (R^2)	16
6.3 Results	17
6.4 Discussion	17
6.5 Discussion of Design Trade-offs.....	18
7. Interactive Application	18
7.1 Analysis Modes.....	18
7.2 Explainability and Manual Override	19
8. Conclusion	19
8.1 Limitations	19
8.2 Future Work	20
9. References.....	20

1. Introduction

Customer relation managements of companies has a direct effect on how happy their customers are, their popularity, and profitability. Quality assurance (QA) is usually done by hand in call centers by their supervisors who listen to a random sample of calls not all of them and then use a checklist to rate each agent's performance. Manually rating calls is slow, expensive, and doesn't always work the same way for each rater. It also only covers a small part of the total call number, so quality problems can go unnoticed for a long time.

The objective of this project is to design an automated, end to end Call Center Quality Scoring System for Turkish-language dialogues. After processing a written transcript or a call recording through a sequential pipeline, the system generates a single quality compliance score on a 0–100 scale that represents the representative's performance. By automating this procedure, the system enables the evaluation of all calls rather than just a small sample, the consistent use of the same scoring criteria, and the identification of low-quality contacts for human review.

The project's methodology combines modern natural language processing (NLP) with speech analytics. A improved Turkish BERT model (BERTurk) provides the basic data-driven regression score, whilst a deterministic, rule-based engine adds interpretability and enforces compliance standards that a holistic model cannot clearly say. To provide empirical support for the final system's design choices, several alternative modeling approaches are also implemented and compared, such as sentence embeddings, a Maximum-Entropy classifier on contextual BERT features, a Log-Likelihood-Ratio (LLR) monitor, and classical bag-of-words baselines.

The main contributions of this project can be summarized as follows:

- A complete, working speech-to-score pipeline for Turkish call recordings, integrating state-of-the-art automatic speech recognition and speaker diarization with downstream natural-language analysis.
- A fine-tuned BERTurk regression model that predicts a continuous 0–100 quality score, together with a systematic empirical comparison against five alternative modeling approaches on an identical test set.
- A transparent, deterministic compliance engine that augments the learned score with auditable checks for greeting, identity verification, empathy, resolution, anomalies (profanity, rude language, shouting) and dead-air silence detected from the audio timestamps.
- A hybrid scoring formula that merges the learned and rule-based signals into a single explainable score, and an interactive web application that exposes the whole system to non-technical quality teams.

The rest of this report is structured as follows. The relevant history and related work are reviewed in Section 2. The dataset's preparation and augmentation are covered in Section 3. Each pipeline level and the overall system design are shown in Section 4. The various modeling techniques and their mathematical formulation are described in depth in Section 5. The experimental design, evaluation criteria, and outcomes including a thorough comparison of all models are reported in Section 6. The interactive application created on top of the system is described in Section 7. The report's conclusion, limits, and future research prospects are covered in Section 8.

2. Background and Related Work

The system draws on four areas of prior work: automated speech recognition, speaker diarization, text representation, and automated call-quality monitoring.

2.1 Speech-to-Text and Diarization

Text is produced from audio using automatic speech recognition (ASR). OpenAI Whisper and other recent transformer-based models offer reliable multilingual transcription, including Turkish. With forced phoneme alignment, word-level timestamps, and a diarization step that establishes identifying speakers speech time, WhisperX expands upon Whisper. Pyannote.audio manages the actual diarization process by segmenting the audio, extracting neural speaker embeddings, and grouping them into discrete speakers. When combined, these technologies produce a time-stamped, speaker labeled transcript from a raw, single-channel recording, which serves as the basis for any subsequent textual analysis.

The word-level timestamps produced by this stage are not merely a by-product; they are essential for two later components of the system. They allow each transcribed word to be attributed to a specific speaker turn, which is what makes role-aware analysis possible, and they enable the measurement of silence between turns, which the rule engine uses to detect dead air. A purely textual transcript without timing information would make these capabilities impossible, which is why an alignment-aware ASR system such as WhisperX was chosen over plain transcription.

2.2 Text Representation

To feed conversations into machine-learning models, text must be converted into numerical features. Classical approaches such as bag-of-words count vectors and Term Frequency–Inverse Document Frequency (TF-IDF) are simple, fast and surprisingly strong baselines, but they ignore word order and context. Distributed word embeddings (e.g., word2vec, fastText) capture semantic similarity but assign a single static vector per word regardless of context. Contextual transformer models such as BERT produce context-dependent representations and, through sub-word tokenization, handle the rich agglutinative morphology of Turkish far better than static

embeddings. BERTurk is a BERT model pre-trained specifically on Turkish corpora, and Sentence-BERT (SBERT) adapts such models to produce sentence-level embeddings.

The choice of representation is especially consequential for Turkish, which is an agglutinative language: a single root can produce a very large number of inflected surface forms by appending suffixes. A static, word-level vocabulary therefore suffers from severe data sparsity and frequently encounters out-of-vocabulary words at test time. Sub-word tokenization, used by BERT and BERTurk, decomposes rare or unseen words into smaller known units, allowing the model to generalize across inflectional variants of the same root. This property is a key reason why a Turkish-specific transformer is well suited to the conversational, morphologically rich text found in call-center transcripts.

2.3 Automated Quality Monitoring

The hybrid scoring architecture of this project follows two classical strands of call-monitoring research. Zweig et al. (2006) framed automated quality monitoring as a Maximum-Entropy (MaxEnt) classification problem, mapping features of a conversation to a quality category. Kim (2007) proposed a Log-Likelihood-Ratio (LLR) monitor that compares each agent utterance against language models of “good” and “bad” calls to produce a turn-by-turn quality signal. These two ideas are reproduced and combined in this work, and are contrasted with a modern fine-tuned transformer regressor.

2.4 Toward an Explainable, Hybrid System

Each of the approaches above has a characteristic weakness when used in isolation. A fine-tuned transformer is the most accurate predictor, but it is a black box: it returns a number without justifying it, which is unsatisfactory for a quality-assurance setting where supervisors must be able to defend a score to an agent and where specific compliance obligations (mandatory greetings, identity verification, regulatory disclosures) must be demonstrably checked. A purely rule-based system, on the other hand, is fully transparent but brittle and unable to capture the nuance of tone, sentiment and coherence that ultimately determines customer satisfaction. The classical statistical monitors of Zweig and Kim sit between these extremes but are comparatively coarse.

This motivates the central design decision of the project: rather than choosing one paradigm, the final system combines a learned model with a deterministic rule engine. The learned model supplies the nuanced, data-driven judgment, while the rule engine supplies the explicit, auditable compliance checks and the explanation of how the final number was reached. This hybrid, explainable design is the main architectural contribution of the work and is detailed in Section 4.

3. Dataset and Preparation

3.1 Dataset Description

The dataset consists of 2,695 Turkish call-center conversations. Each record contains the full conversation text (with each turn labeled as representative or customer), a ground-truth quality score in the range 0–100 (`final_quality_score`), and an expected quality class. The conversations are labeled into three quality classes — Good (1,295 conversations), average (700) and poor (700). The mean ground-truth score is 55.4 with a standard deviation of 28.1, indicating that the dataset spans the full quality spectrum rather than clustering around a single value. Figure 1 shows the class distribution and Figure 2 the distribution of the continuous quality scores.

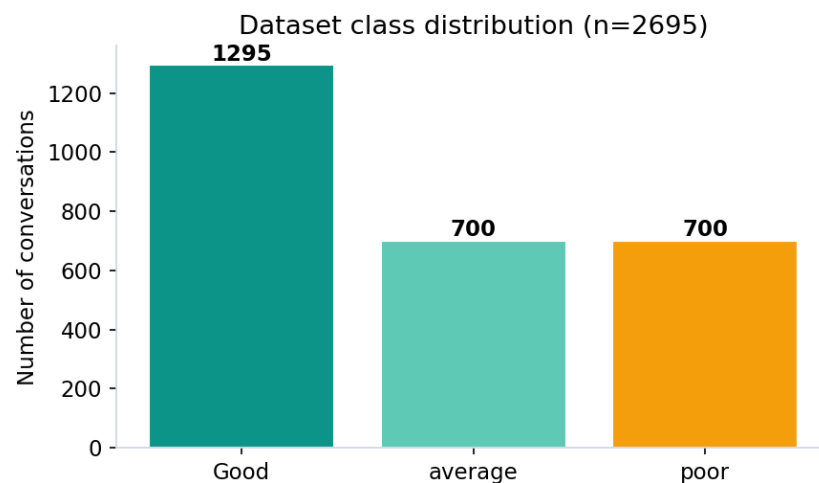


Figure 1: Distribution of conversations across the three quality classes.

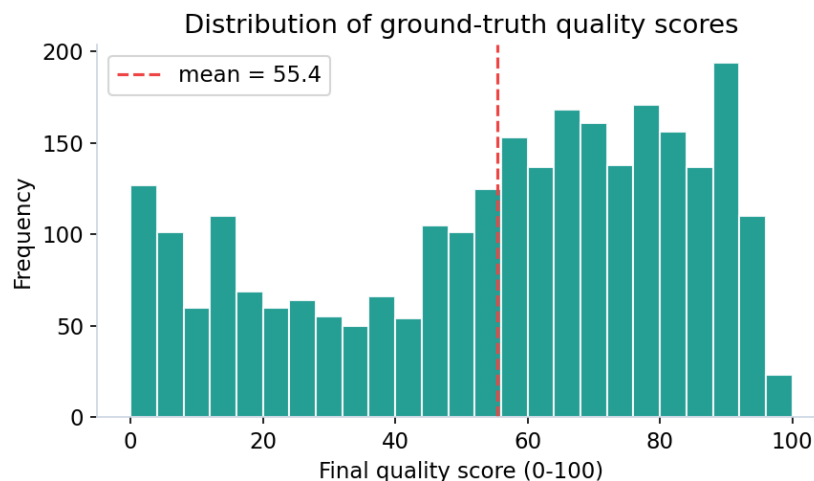


Figure 2: Histogram of the ground-truth quality scores (0–100).

3.2 Data Augmentation

Because annotated call-center data is scarce and the model must generalize to varied phrasing, the training portion of the data was expanded with augmented conversations. Augmented samples are linked to their base conversation through an identifier so that augmentation is applied only to the training split and never leaks into the test split, which would otherwise inflate the reported performance. This augmentation increases the diversity of surface realizations seen during training while preserving the original held-out test set for fair evaluation.

3.3 Preprocessing and Speaker Mapping

For audio inputs, the recording is first transcribed and diarized, producing anonymous speaker labels (SPEAKER_00, SPEAKER_01). These labels must be mapped to the semantic roles representative and customer before scoring. The system performs this mapping automatically using a weighted-signal heuristic: each speaker receives a “representative score” computed from greeting and opening phrases, identity-verification questions, company self-introduction, offers of help, resolution language, courtesy expressions and the number of questions asked, while customer signals (problem statements, negative emotion) reduce the score. The speaker who opens the call also receives a small bias, since the agent typically begins the conversation. The highest-scoring speaker is assigned the representative role. If the automatic assignment is incorrect, the user can override it manually. The resulting text is normalized into the canonical “role: utterance” format expected by all downstream models.

This weighted heuristic is deliberately preferred over a single naive rule such as “the first speaker is the agent”. Although the agent usually opens the call, exceptions occur for example when the customer answers an outbound call or speaks first to confirm the line and a single-signal rule fails in those cases. By accumulating evidence from several role-indicative signals, the heuristic remains correct even when the customer happens to greet first, because the genuine agent still accumulates more weight from company self-introduction, identity questions and resolution language.

3.4 Quality Score and Label Definition

Two related targets are derived from the data. The regression target is the continuous `final_quality_score` in the range 0–100, which all regression models (including the fine-tuned BERTurk model and the baselines) are trained to predict. The classification target is the discrete `expected_quality_class`, taking the values Good, average or poor, which is used to train the MaxEnt classifier and the LLR language models. The continuous score is the primary, finer-grained signal, while the class labels provide a coarser, categorical view of the same underlying quality.

For reporting and decision-making, the continuous score is additionally mapped to a four-level quality band: a score of 85 or above is reported as Excellent, 70–84 as Good, 55–69 as Average, and below 55 as Poor. A separate operational threshold flags any call scoring below 60 for

mandatory manual review, ensuring that low-quality interactions are escalated to a human supervisor rather than silently archived. These thresholds belong to the reporting layer only and do not alter the underlying model output.

4. System Architecture

The system implements a sequential pipeline in which the output of each stage becomes the input of the next, as illustrated in Figure 3. The stages are: (1) speech-to-text and diarization, (2) preprocessing and speaker-role mapping, (3) NLP analysis with the BERTurk regression model, (4) the deterministic rule engine, and (5) the hybrid scoring engine that combines them into a final 0–100 score.

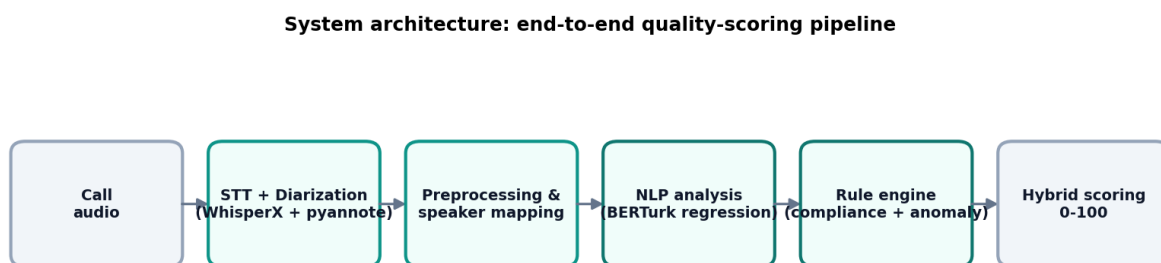


Figure 3: End-to-end architecture of the quality-scoring pipeline.

4.1 Speech-to-Text and Diarization

Audio uploaded to the system is transcribed with WhisperX and diarized with pyannote.audio. To remain robust across file formats (wav, mp3, m4a, mp4), the already-decoded 16 kHz mono waveform produced during transcription is passed directly to the diarization model, avoiding format-specific decoding errors. Diarization requires authenticated access to gated pyannote models via a Hugging Face token. The output is a list of time-stamped segments, each annotated with text and a speaker label.

4.2 Automatic Speaker-Role Assignment

As described in Section 3.3, the anonymous diarization labels are converted into representative and customer roles by a transparent, weighted-signal classifier. This component is deliberately rule-based rather than learned, both because labeled role data is unavailable and because the heuristic is interpretable and easily corrected by a human reviewer.

4.3 NLP Analysis with BERTurk

The core data-driven component is a BERTurk model fine-tuned as a regressor. The cleaned transcript is tokenized and passed through the model, which outputs a normalized value that is scaled to the 0–100 range to yield the model quality score. This component captures holistic,

learned notions of quality tone, coherence and customer satisfaction that are difficult to encode as explicit rules.

4.4 Rule-Based Engine

The rule engine encodes the explicit compliance criteria of a call-center quality scorecard as deterministic checks evaluated on the representative's utterances. The checks are grouped into five categories: Opening (greeting, company/self introduction, offer of help), Identity Verification, Empathy and Courtesy, Resolution and Closing (concrete action offered, further-needs question, proper farewell), and Anomaly detection (profanity, rude or dismissive language, shouting). A sixth category, Silence / dead air, is available for audio inputs and uses the segment timestamps to detect long silences during which the agent leaves the customer unattended; a silence preceded by a hold announcement ("one moment, I am checking") is penalized only lightly, whereas unannounced dead air is penalized heavily. Each satisfied positive check contributes a small reward, while violations and anomalies subtract points; critical violations such as profanity additionally impose a hard cap on the final score. Figure 4 summarizes the maximum reward and penalty associated with each category.

Concretely, the engine evaluates the following groups of checks on the representative's turns:

- Opening: whether the agent greeted the customer, introduced the company and themselves, and offered help (e.g., "how may I help you?").
- Identity verification: whether the agent requested an identifying detail (customer number, identity number) before discussing the account.
- Empathy and courtesy: whether the agent used empathetic phrases ("I understand", "you are right", "I apologize") and courteous language; a stronger penalty applies when the customer expressed negative emotion but no empathy was shown.
- Resolution and closing: whether a concrete action or solution was offered, whether the agent asked if anything else was needed, and whether the call ended with thanks and a proper farewell.
- Anomaly: detection of profanity, rude or dismissive language, and shouting markers; these contribute no reward and impose large penalties, with profanity additionally capping the final score.
- Silence / dead air: for audio inputs, long silences are detected from the segment timestamps; an unannounced silence is penalized heavily, whereas a silence preceded by a hold announcement is penalized only lightly.

Each satisfied positive check adds a small reward and each violation subtracts a penalty, so that the rule contribution to the score is symmetric and fully traceable. Figure 4 visualizes the maximum reward and penalty for each category.

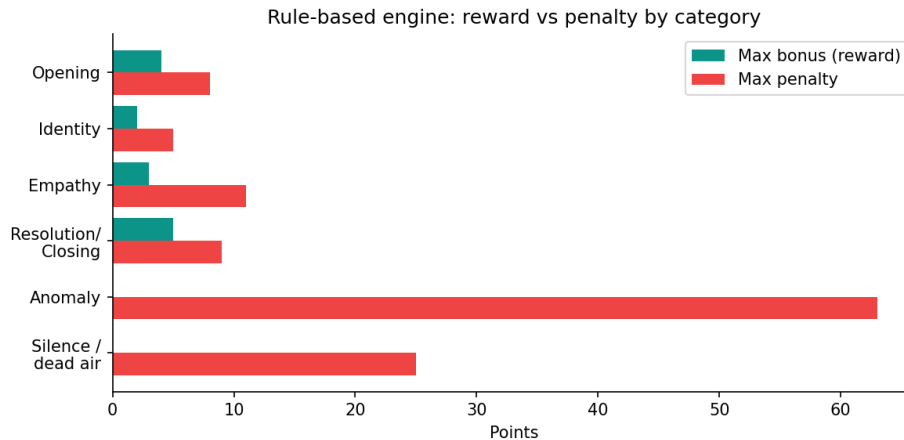


Figure 4: Maximum reward (bonus) and penalty per rule-engine category.

4.5 Hybrid Scoring Engine

The final score combines the learned model score with the deterministic rule engine. The BERTurk regression score acts as the anchor; the rule engine then adds bonuses for satisfied criteria, subtracts penalties for violations, and applies a cap when a critical anomaly is present, as shown in Equation (1). The result is clipped to the valid 0–100 range.

$$final_score = clip(model_score + bonus_total - penalty_total, 0, 100), \text{ subject to anomaly cap} \quad (1)$$

This design preserves the predictive power of the transformer while guaranteeing that explicit, auditable compliance criteria are reflected in the score. It also makes the result explainable: the interface shows exactly which criteria passed or failed and how many points each contributed, which directly answers the central QA question “why did this call receive this score?”. A further consequence of the reward mechanism is that a fully compliant, high-quality call can reach the maximum score of 100 even though the regression model alone is conservative and rarely outputs extreme values; the bonuses lift an already strong model score to the ceiling, while a weak model score is not pushed to 100 by compliance alone.

4.6 Implementation and Engineering Challenges

Integrating several rapidly evolving deep-learning libraries into a single reproducible pipeline raised a number of practical engineering challenges that are worth documenting, as they are typical of real speech-and-NLP systems. The most significant was dependency management: installing the speech stack without pinned versions pulled in mutually incompatible releases of PyTorch, the transformers library and the Hugging Face Hub client, producing a cascade of import and runtime errors. The problem was resolved by fixing a coherent, mutually compatible set of versions in a locked requirements file, which is the recommended way to guarantee reproducibility of such a system.

Several format and compatibility issues were also handled at the code level. A change in the default behaviour of the model-loading routine in recent PyTorch versions required explicit handling so that the diarization checkpoints could be loaded. The diarization model could not directly read certain compressed audio formats such as m4a, which was solved by feeding it the already-decoded waveform produced during transcription rather than the original file. Finally, the diarization models are access-gated and require an authenticated token together with acceptance of their licenses; the application therefore manages the token securely and fails gracefully with a clear message when diarization is unavailable, falling back to single-speaker analysis.

5. Modeling Approaches

To justify the choice of the final architecture, several modeling approaches were implemented and evaluated on an identical train/test split. They range from classical bag-of-words regressors to the fine-tuned transformer and the MaxEnt+LLR hybrid.

5.1 Baseline Regressors

Three baseline regressors were trained to predict the quality score directly from text. The first uses TF-IDF features (unigrams and bigrams) with a Ridge regression head. The second replaces TF-IDF with raw count vectors. The third encodes each conversation with a multilingual Sentence-BERT model and fits a Ridge regressor on the resulting sentence embeddings. These baselines are fast, require no GPU and establish the level of performance achievable with conventional methods.

5.2 BERTurk Fine-Tuned Regression

The principal model is the BERTurk transformer fine-tuned end-to-end for the regression task. Unlike the baselines, which use fixed features, fine-tuning adapts every layer of the network to the specific objective of predicting call quality, allowing it to model context, tone and long-range dependencies in the conversation.

5.3 Maximum-Entropy Classifier

Following Zweig et al. (2006), a Maximum-Entropy classifier was implemented as a multinomial logistic regression over the [CLS] embedding extracted from a (non-fine-tuned) BERTurk model. The MaxEnt model assigns a probability to each quality class, as in Equation (2), where the f_i are the BERT features and the λ_i are the learned weights.

$$P(\text{class} | C) = (1 / Z) \cdot \exp(\sum_i \lambda_i \cdot f_i(\text{class}, C)). \quad (2)$$

A continuous score is then derived from the class probabilities by a probability-weighted combination of class anchor scores (Good = 90, average = 55, poor = 15), as in Equation (3).

$$\text{score} = P(\text{Good}) \cdot 90 + P(\text{average}) \cdot 55 + P(\text{poor}) \cdot 15 \quad (3)$$

5.4 Log-Likelihood-Ratio Monitor

Following Kim (2007), two bigram language models with Laplace smoothing are trained on the representative utterances of Good calls and of Poor calls respectively. For each new conversation, the log-likelihood ratio of each agent utterance under the two models is accumulated, as in Equation (4). A persistently negative cumulative ratio indicates a low-quality interaction; the cumulative value is normalized to a 0–100 score. The original design also raises a turn-by-turn alert when the cumulative ratio crosses a threshold, providing an early-warning signal that a holistic model cannot offer.

$$LLR(x) = \log(P(x | LM_{good}) / P(x | LM_{bad})) \quad (4)$$

5.5 MaxEnt + LLR Hybrid

The two classical signals are combined into a single hybrid score as a weighted sum, Equation (5), giving more weight to the stronger MaxEnt component. This reproduces the hybrid architecture of the design document and serves as a fully interpretable alternative to the fine-tuned transformer.

$$final_score = 0.70 \cdot maxent_score + 0.30 \cdot llr_score \quad (5)$$

5.6 Implementation Details

To make the comparison fair and reproducible, all models were trained and evaluated under a common protocol with explicit hyperparameters. The key settings of each model are summarized below:

- TF-IDF + Ridge: TF-IDF features over unigrams and bigrams with a vocabulary capped at 20,000 terms, followed by Ridge regression with regularization strength $\alpha = 1.0$.
- CountVectorizer + Ridge: identical configuration to the TF-IDF model but using raw term counts instead of TF-IDF weighting.
- Sentence-BERT + Ridge: each conversation is encoded with a multilingual paraphrase Sentence-BERT model, and a Ridge regressor ($\alpha = 1.0$) is fit on the resulting fixed-length embeddings.
- BERTurk Fine-Tuned: the dbmdz/bert-base-turkish-cased transformer fine-tuned end-to-end for the regression objective, with the output scaled to the 0–100 range.
- MaxEnt: multinomial logistic regression on standardized [CLS] embeddings from a (non-fine-tuned) BERTurk model, with balanced class weights and the L-BFGS solver.
- LLR Monitor: two bigram language models with Laplace (add-one) smoothing, trained on the agent utterances of Good and Poor calls respectively.

Feature scaling was applied where appropriate in particular, the [CLS] features used by the MaxEnt model were standardized to zero mean and unit variance to ensure stable convergence

of the logistic-regression solver. The fine-tuned transformer and the live extraction of BERT embeddings were executed on a GPU when available and on the CPU otherwise.

6. Experiments and Results

6.1 Experimental Setup

All models were tested using the same held-out test set (80/20 train/test split stratified by quality class) with a fixed random seed (17) such that each model is evaluated on the same data. Data augmentation was used only on the training split and never on the test split. Therefore no augmented version of a test conversation can leak into training and artificially inflate the reported performances.

6.2 Evaluation Metrics

The models are evaluated as regressors since the goal quality score is a continuous number in the range 0-100, not as classifiers. We report three unique regression metrics each capturing a different aspect of the prediction error such that no single number covers a model flaw. In the following definitions, the symbols can be interpreted as: y_i the true (ground-truth) score of conversation i , $\hat{y}_i$ the score predicted by the model for that conversation, $\bar{y}$ the mean of the true scores over the entire test set, and n the number of conversations in the test set.

6.2.1 Mean Absolute Error (MAE)

The Mean Absolute Error is the average of the absolute differences between the predicted scores and the true scores, as defined in Equation (6). Since it is expressed in the same unit as the target variable, namely points on the 0–100 quality scale, it is easy to interpret. It shows how many points, on average, the model prediction differs from the true quality score. For example, an MAE of 7 means that the predicted score is approximately 7 points away from the true score across the test calls. MAE treats all errors linearly, so it is less affected by a small number of very large errors compared to squared-error metrics such as RMSE.

.

$$MAE = (1 / n) \cdot \sum |y_i - \hat{y}_i|. \quad (6)$$

where:

y_i : true quality score of conversation i

$\hat{y}_i$: predicted quality score of conversation i

n : number of conversations in the test set

6.2.2 Root Mean Squared Error (RMSE)

The Root Mean Squared Error measures the average prediction error after giving larger weight to bigger mistakes. It is calculated by squaring each error, taking the average of these squared errors, and then applying the square root, as shown in Equation (7). Since the final value is again in the original 0–100 score unit, it can still be interpreted in terms of quality-score points. Compared with MAE, RMSE is more sensitive to large individual errors. Therefore, if RMSE is much higher than MAE, it usually means that the model makes reasonable predictions for many calls but produces some large mistakes on specific examples. In this project, RMSE was useful for checking whether the model had such severe prediction errors, not only its average performance.

$$RMSE = \sqrt{(1/n) \cdot \sum (y_i - \hat{y}_i)^2} \quad (7)$$

where:

y_i : true quality score of conversation i

$\hat{y}_i$: predicted quality score of conversation i

n : number of conversations in the test set

6.2.3 Coefficient of Determination (R^2)

The Coefficient of Determination, R^2 , measures the proportion of the variance in the true scores that is explained by the model, as defined in Equation (8). It compares the model's total squared error (the numerator) against the squared error of a trivial baseline that always predicts the mean score $\bar{y}$ (the denominator). A value of $R^2 = 1$ indicates perfect prediction, $R^2 = 0$ indicates performance no better than always predicting the mean, and a negative R^2 indicates a model that performs worse than that trivial baseline. Unlike MAE and RMSE, R^2 is dimensionless; this scale-free property makes it convenient for comparing models across different datasets and score ranges.

$$R^2 = 1 - (\sum (y_i - \hat{y}_i)^2) / (\sum (y_i - \bar{y})^2) \quad (8)$$

where:

y_i : true quality score of conversation i

$\hat{y}_i$: predicted quality score of conversation i

$\bar{y}$: mean of the true scores over the test set

Reporting all three metrics together provides a complete picture of model quality: MAE gives an intuitive average error in score points, RMSE exposes sensitivity to large errors, and R^2 provides

a scale-free measure of how well the model captures the overall variation in call quality. A strong model is expected to have low MAE and RMSE together with a high R^2 .

6.3 Results

Table 1 reports the performance of all seven models, sorted from best to worst by MAE, and Figure 5 visualizes the same comparison across the three metrics.

Table 1: Model performance on the held-out test set (sorted by MAE).

Model	MAE	RMSE	R^2
BERTurk Fine-Tuned Regression	7.35	10.21	0.876
TF-IDF + Ridge Regression	9.83	12.95	0.785
CountVectorizer + Ridge Regression	11.05	14.37	0.755
Sentence-BERT + Ridge Regression	13.48	17.23	0.647
MaxEnt + LLR Hybrid	15.49	20.48	0.458
MaxEnt Classifier (BERT features)	15.54	21.50	0.403
LLR Monitor	28.78	35.62	-0.640

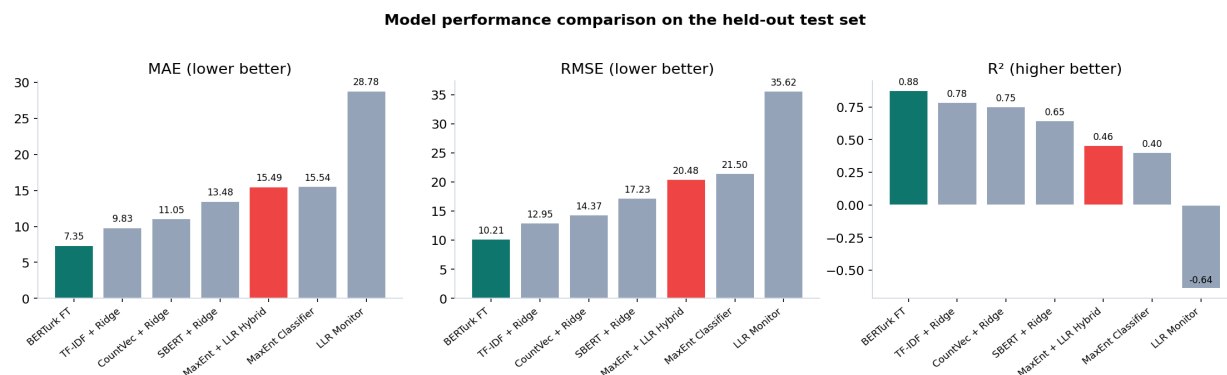


Figure 5: Comparison of all models across MAE, RMSE and R^2 .

6.4 Discussion

The fine-tuned BERTurk regressor is the clear best performer, with an MAE of 7.35, an RMSE of 10.21 and an R^2 of 0.876, substantially ahead of every other approach. This confirms that task-specific fine-tuning of a Turkish transformer is the most effective way to predict the continuous quality score. Among the baselines, TF-IDF + Ridge is remarkably strong (MAE 9.83, R^2 0.785), outperforming both the count-vector and the Sentence-BERT baselines; this is a common finding, as raw [CLS]/sentence embeddings without task fine-tuning are often weaker features than well-tuned TF-IDF for a focused regression task.

Although MaxEnt and LLR are not as good as the primary regression models, they nevertheless helped to comprehend the system architecture. The error values were higher than not powerful enough to utilize as the final scoring model. But they were more interpretable than a fine-tuned transformer. The LLR monitor was the least effective model. This is not surprising as a bigram likelihood ratio can only offer a restricted perspective of the dialog. Based on these results, I utilized BERTurk as the main scoring model, further enriching it with a basic rule-based layer to increase explainability. This rule layer analyzes observable quality signals in the representative's turns, such as greeting, apology, empathy, offer of aid and closing sentences.

6.5 Discussion of Design Trade-offs

The trial ranking shows a common trade-off between being correct and being easy to understand. The fine-tuned transformer is the most accurate but also the least clear. The MaxEnt and LLR models, on the other hand, are much easier to understand but not nearly as accurate. The project doesn't see this as a black or white choice, which is very important. The accurate transformer is used as the score anchor, and the deterministic rule engine is added on top. This gives the end system most of the accuracy of the transformer while also adding the transparency and compliance guarantees that the transformer lacks on its own. In other words, the design theory was based on the weaker interpretable models, even though the strongest single model was chosen as the main predictor.

In this experiment, TF-IDF gave better results than Sentence-BERT. This was mainly because Sentence-BERT was not fine-tuned for this specific task. It could understand the general meaning of the conversations, but the quality score often depended on smaller details in the wording. For example, phrases related to greeting, apology, empathy, and offering a solution were important signals in the dataset. TF-IDF captured these repeated words and phrases more directly. Because of this, the simpler TF-IDF model worked better than the more general Sentence-BERT embeddings in this case.

.

7. Interactive Application

The entire system is provided as an interactive online application, developed with Streamlit. The application converts the underlying models into a usable tool for a non-technical quality team to use on real calls and is the main deliverable via which the project is demonstrated.

7.1 Analysis Modes

There are two analysis types available in the app. In text mode, a reviewer can paste a labeled transcript and get a score right away. This is helpful for quickly testing how the scores are calculated or for talks that have already been typed up. In audio mode, a reviewer uploads a call recording in wav, mp3, m4a, or mp4 format, and the whole speech pipeline runs instantly from

start to finish. The audio is transcribed, the speakers are identified and assigned to roles, and the conversation is scored. All of this can be done with a single click. During the possibly time-consuming transcription step, a spinner shows progress.

7.2 Explainability and Manual Override

After analysis, the interface shows the end quality score, the quality level, a flag for manual review, a progress bar, and a short evaluation in text form. Below these cards with summaries, there is a clear breakdown that shows each rule-engine check, grouped by category, marked as passed or failed and with how many points it added to the final score. For audio sources, there is a short silence summary that shows the total silence and the number of dead-air events. This breakdown makes the system's goal of being explainable a reality; a supervisor can see at a peek not only the score but also the exact reasons why it was given.

Automatically assigning speakers to roles can go wrong sometimes, so an advanced panel lets the reviewer look at the raw speaker-labeled text and fix the role mapping. The call is then re-scored with the new roles. These "human-in-the-loop" controls make the automatic system reliable without slowing it down when the heuristic is right, which is most of the time..

8. Conclusion

In this project an end-to-end automatic quality-scoring system for Turkish call-center conversations has been designed and implemented. The audio is transcribed and diarized and speakers are assigned to roles. The conversation is scored with a fine-tuned BERTurk regressor and the result is improved with a transparent rule-based engine that applies explicit compliance criteria and detects anomalies such as profanity and dead air. We combine the learned and rule-based signals into a single explainable score between 0-100 using a hybrid scoring formula.

Empirically, the fine-tuned BERTurk model achieved the best regression performance (MAE 7.35, R^2 0.876), clearly surpassing TF-IDF, count-vector and Sentence-BERT baselines as well as the MaxEnt classifier and LLR monitor. The classical MaxEnt and LLR methods, while less accurate, contributed interpretability and a turn-by-turn alerting capability, which justified the hybrid, rule-augmented design of the final system.

The outputs of this study can be used by call centers to automatically score each call, enforce consistent compliance criteria and prioritize low-quality interactions for human review, replacing the slow and partial manual sampling used today. The system also enables performance tracking over time and the identification of systematic quality issues that random sampling would miss, by scoring the entire volume of calls, not a small sample.

8.1 Limitations

Some limitations need to be discussed. First, the anomaly detection for shouting is currently based on textual cues, which are unreliable because automatic transcripts do not faithfully

preserve loudness or emphasis, and genuine shouting is better detected from the acoustic signal. Second, the quality of the downstream score depends on the quality of the transcription and diarization, such that noisy recordings, strong accents or overlapping speech can impair the final result. Third, the rule engine relies on curated Turkish keyword lists for greetings, empathy, profanity and hold announcements. These lists are extensible but cannot capture every phrasing and may need tuning for a given call center. The ground-truth scores used for training and evaluation are the scores produced by the dataset's own labeling process and have not been independently calibrated against multiple human reviewers.

8.2 Future Work

These limitations naturally suggest future work. Anomaly detection at the acoustic level would enable true shouting, interruptions and long overlaps to be detected from the audio signal rather than from text. Better accuracy and trust in the score could be achieved by expanding and diversifying the labeled dataset and calibrating the model score against a panel of human reviewers. The rule engine could be extended as a configurable, client-specific framework in which each call center defines its own mandatory scripts, disclosures and weightings without changing code. Other directions include real-time scoring of live calls using the turn-by-turn LLR signal as an early-warning mechanism, multilingual support beyond Turkish, and automatic coaching feedback generated for agents from the per-category breakdown produced by the rule engine.

9. References

- [1] Zweig, G., Siohan, O., Saon, G., Ramabhadran, B., Povey, D., Mangu, L., & Kingsbury, B. (2006). Automated quality monitoring for call centers using speech and NLP technologies. Proceedings of HLT-NAACL.
- [2] Kim, W. (2007). Online call quality monitoring using language models. Interspeech.
- [3] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL-HLT.
- [4] Schweter, S. (2020). BERTurk – BERT models for Turkish. Zenodo.
<https://doi.org/10.5281/zenodo.3770924>
- [5] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision (Whisper). ICML.
- [6] Bain, M., Huh, J., Han, T., & Zisserman, A. (2023). WhisperX: Time-accurate speech transcription of long-form audio. Interspeech.
- [7] Bredin, H., et al. (2020). pyannote.audio: Neural building blocks for speaker diarization. ICASSP.
- [8] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP-IJCNLP.

- [9] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.